

Astro Intro

→ What Is Astro ?

> *Astro is a JavaScript web framework optimized for building fast, content-driven websites.*

Target use cases

- Blogs & personal sites
- Documentation sites
- Marketing & landing pages
- Portfolios & e-commerce
- News & publishing sites

> 📖 https://astro.build 🔗

Not the best fit for

- Dashboards with real-time data
- Apps with heavy user interaction (CRUD)
- Authenticated user sessions
- SaaS applications

→ The JavaScript Problem

Traditional SPA (React, Vue…)

- Ships **entire JS bundle** to browser
- Even mostly-static pages hydrate fully
- More JS → slower load → worse UX
- More JS → harder for search crawlers → worse SEO
- Users pay the cost even for content they just read

Astro's

- **Zero JS by default** → static HTML shipped
- JS only where you explicitly ask for it
- Server renders everything it can
- Client gets lean, fast HTML

→ Design Principles

#	Principle	What it means
1	**Content-Driven**	Built for content delivery, not complex app logic
2	**Server-First**	Render on server by default (like traditional web frameworks)
3	**Fast by Default**	Performance is automatic, not something you optimise later
4	**Easy to Use**	Template syntax is a superset of HTML (any HTML is valid Astro)
5	**UI-Agnostic**	Use React, Vue, Svelte, Solid, or none (on the same page)

→ .Astro File Anatomy

Component Script

```
---
// Runs on SERVER only
// Never sent to the browser
import Layout from '../layouts/Layout.astro';
import { getCollection } from 'astro:content';

const posts = await getCollection('blog');
const { title } = Astro.props;
---
```

- Fetch data, import components, define variables
- Any `await` works here (it's server-side)
- Stripped from final HTML sent to user

Component Template

```
<Layout title={title}>
  <h1>{title}</h1>
  <ul>
    {posts.map(p => (
      <li><a href={p.slug}>{p.data.title}</a></li>
    ))}
  </ul>
</Layout>

<style>
  h1 { color: #ff5d01; } /* scoped by default */
</style>
```

- HTML + JavaScript expressions `{ }`
- `<style>` is **scoped** to this component
- No React - just HTML-like syntax

→ Routing & Project Structure 📁

```
src/
  pages/              ← file-based routing
    index.astro       →  /
    about.astro       →  /about
    blog/
      index.astro     →  /blog
      [slug].astro    →  /blog/:slug  (dynamic)
  components/         ← reusable .astro files
  layouts/            ← shared page templates
  content/            ← markdown content collections
public/               ← static assets (images, fonts)
astro.config.mjs      ← configuration
```

Same mental model as Next.js

- File = Route ✅
- `[slug].astro` = dynamic segment ✅
- `public/` = static assets ✅

Key difference

- Next.js: `app/page.tsx`
- Astro: `src/pages/index.astro`

```
npm run dev    # localhost:4321
npm run build  # → dist/
npm run preview
```

→ Layouts & Slots

Define a Layout

```
---
// src/layouts/BaseLayout.astro
const { title } = Astro.props;
---
<html lang="en">
  <head>
    <title>{title}</title>
  </head>
  <body>
    <nav><!-- shared navigation --></nav>

    <main>
      <slot />  <!-- page content goes here -->
    </main>

    <footer>
      <slot name="footer" />  <!-- named slot -->
    </footer>
  </body>
</html>
```

Use a Layout in a page

```
---
import BaseLayout from '../layouts/BaseLayout.astro';
---
<BaseLayout title="Home Page">
  <h1>Welcome!</h1>
  <p>This is injected into the slot.</p>

  <p slot="footer">Footer content here.</p>
</BaseLayout>
```

- Same concept as Next.js `layout.tsx`
  - shared UI across pages without repetition.

→ Markdown & Content Collections

Markdown pages

Any `.md` file in `src/pages/` becomes a route:

```
---
layout: ../layouts/BlogLayout.astro
title: "My First Post"
author: "Jane"
date: 2025-04-20
---

# My First Post

Write your content in **Markdown**.
Astro handles the rest.
```

> Frontmatter data passed to the layout as props.

Content Collections

Organised, validated, typed content:

```ts
// src/content/config.ts
import { defineCollection, z } from 'astro:content';

export const collections = {
  blog: defineCollection({
    schema: z.object({
      title: z.string(),
      date: z.date(),
      author: z.string(),
    }),
  }),
};
```

```ts
// In any .astro page
import { getCollection } from 'astro:content';
const posts = await getCollection('blog');
// posts → typed, validated array
```

→ Islands Architecture

The concept

- Most of page = **static HTML** (fast, cacheable, SEO-friendly)
- **"Islands"** = isolated interactive components, each hydrates independently
- No monolithic JS bundle — only the code each island needs

> Islands → render HTML pages on the server, and inject placeholders around highly dynamic regions that can then be hydrated on the client

Two island types

Client Islands - interactive in the browser

- Hydrated with `client:*` directives
- Each island is independent (no bottlenecks)

Server Islands - dynamic server content

- Rendered server-side on request
- Unblocks the static page shell
- Used for personalised content (user data, recommendations)

→ Islands: Hydration Directives

Client Islands `client:*`

```
<!-- Hydrate immediately on page load -->
<SearchBar client:load />

<!-- Hydrate when component enters viewport -->
<VideoPlayer client:visible />

<!-- Hydrate when browser is idle -->
<Analytics client:idle />

<!-- Works with any UI framework! -->
<ReactShoppingCart client:load />
<VueFeedWidget client:visible />
<SvelteCounter client:idle />
```

> Each island loads its JS **independently** (..others don't wait).

Server Islands `server:defer`

```
<!-- Render this part server-side, deferred -->
<!-- Page shell loads immediately -->
<!-- This slot loads separately -->
<UserAvatar server:defer>
  <!-- Fallback shown while loading -->
  <GenericAvatar slot="fallback" />
</UserAvatar>

<PersonalisedDeals server:defer>
  <StaticDeals slot="fallback" />
</PersonalisedDeals>
```

> Static page shell renders instantly.
> Personalised parts fill in after.

→ UI Framework Integration

Astro is UI-agnostic → use any framework, or none at all.

Add integrations

```
# Add React support
npx astro add react

# Add Vue support
npx astro add vue

# Add Tailwind
npx astro add tailwind

# Multiple at once
npm create astro@latest -- --add react --add tailwind
```

Mix frameworks on one page

```
---
// Mix React, Vue, Svelte
import ReactButton from './Button.jsx';
import VueCard from './Card.vue';
import SvelteToggle from './Toggle.svelte';
---

<!-- Each runs ONLY where client:* is placed -->
<ReactButton client:load />
<VueCard client:visible />
<SvelteToggle client:idle />

<!-- Everything else = static HTML -->
<h1>This is just HTML. Zero JS.</h1>
```

> Each framework only runs where you put `client:*` — not globally. You pay the JS cost only for what you actually need.

→ Installing Astro

Quick start

```
npm create astro@latest
```

Interactive wizard asks:

- Project name
- Starter template (blog, docs, empty…)
- TypeScript? (recommended)
- Install dependencies?

```
# Or start from a template
npm create astro@latest -- --template blog
npm create astro@latest -- --template docs
```

> Node.js v22.12.0 or higher required.

`package.json` scripts

```
{
  "scripts": {
    "dev":     "astro dev",
    "build":   "astro build",
    "preview": "astro preview"
  }
}
```

- `dev` → dev server at **`localhost:4321`** with hot reload
- `build` → optimised static output in `dist/`
- `preview` → serve the built `dist/` locally

→ Next.Js Vs Astro

	Next.js	Astro
Default rendering	SSR / React Server Components	SSG — zero JS by default
Best for	Apps, auth, real-time data, CRUD	Blogs, docs, marketing, portfolios
Components	React everywhere	`.astro` + React / Vue / Svelte / any
Hydration	All `'use client'` components hydrated	Explicit islands only (`client:*`)
Data fetching	Server Components / Server Actions	Frontmatter `---` at build time
Routing	`app/` folder (`page.tsx`)	`src/pages/` folder (`index.astro`)
JS bundle size	Moderate to large	Near zero
Dev server	`localhost:3000`	`localhost:4321`
Learning curve	Requires React knowledge	HTML + JS is enough

→ When To Choose What ?

Choose Next.js when...

- User **authentication** is required
- **Real-time** or frequently updated data
- Complex **CRUD forms** and mutations
- **Dashboard** or SaaS application
- Heavy client-side **interactivity**
- **API routes** alongside the frontend

Choose Astro when...

- Content is mostly **static**
- **SEO** is a top priority
- **Performance** matters above all
- Building a **blog**, docs, portfolio, or marketing site
- You want to mix UI frameworks
- Minimal JS payload is important

*Same architectural decision as choosing REST vs GraphQL, or microservices vs monolith → **right tool for the job**.*

Ερωτήσεις ?

Resources

- 🔖 https://astro.build Official site
- 📖 https://docs.astro.build Documentation
- https://docs.astro.build/en/tutorial/0-introduction/ Build a Blog tutorial
  - https://github.com/withastro/blog-tutorial-demo Finished Tutorial repo